OScaR vs TurboQuant

KV Cache Compression Codec Comparison Report

Generated by dhurandhar — May 2026

Executive Summary

This report compares two KV cache compression codecs for edge LLM deployment: TurboQuant (randomized Hadamard rotation + sign quantization + residual int-quant) and OScaR (Omni-Scaled Canalized Rotation — the same Hadamard, plus per-token L2 normalization and groupwise INT for keys). OScaR is designed specifically to address *Token Norm Imbalance* (TNI): the failure mode where a few large-norm tokens dominate the per-tensor quantization scale and push the rest of the sequence under the quant noise floor.

Headline result: TurboQuant wins at equal nominal bit budgets on both regimes. On generic heavy-tail KV (per-channel outliers, uniform token norms), OScaR's MSE at 4-bit is **51–179% worse** than TurboQuant (median 105% worse). On explicit TNI KV (5% of tokens scaled 10×), OScaR's MSE is **33–122% worse** than TurboQuant (median 67% worse) — better than on the uniform distribution, but still behind.

The design intuition is partially validated. OScaR closes the gap by roughly **38 percentage points** between regimes (median 105% → 67% deficit). The per-token normalization does help on TNI data, but not enough to overtake TurboQuant.

Why TurboQuant wins. Comparing at the same nominal bit budget is unfair to OScaR: TurboQuant's per-channel representation is *1 sign bit + N residual bits* (≈ N + 1 effective bits with the sign capturing the dominant post-Hadamard direction), while OScaR is just N bits of groupwise INT. After the Hadamard, the sign-quantization step is already an excellent approximation, leaving a small residual to quantize. OScaR has to quantize the full magnitude, which needs more bits to match TurboQuant's effective precision. This structural advantage explains the consistent gap across bit budgets and head_dim values.

Practical implication. OScaR's main appeal — no calibration, no PCA, simple kernel structure — is real, but the quality/bit tradeoff is worse than TurboQuant on synthetic data. Real-model activations may differ, especially post-RoPE on models with attention sinks; measure before choosing. To close the gap, OScaR would need to adopt TurboQuant's sign+residual structure (orthogonal to its omni-token scaling).

Methodology

Both codecs are tested on the same KV tensor per model (seq_len=1024, num_kv_heads and head_dim per model). Two synthetic distributions are used:

- **Heavy-tail (no TNI):** Gaussian + 5% outlier-channel mixture. Per-channel heavy tails, but per-token norms stay uniform. This is the standard benchmark used in the TurboQuant paper.
- **Token Norm Imbalance:** the same heavy-tail base, with an additional per-token scaling — 5% of tokens are multiplied by 10×, simulating attention-sink tokens. This exhibits the exact failure mode OScaR's design targets.

Quality is measured by cosine similarity and MSE on a full compress→decompress round-trip. Reported MSE reduction = (1 – OScaR_MSE / TQ_MSE) × 100%. Positive means OScaR wins; negative means TurboQuant wins.

TurboQuant pipeline:

- Randomized Hadamard rotation via dense matmul — $O(d^2)$ (spreads per-channel outliers uniformly)
- Sign quantization (1 bit/dim) + per-vector L2 norm
- Uniform residual correction at configured bit precision

OScaR pipeline:

- Same randomized Hadamard rotation (canalized rotation step)
- Omni-token scaling: divide out each token's post-rotation L2 norm and store as 16-bit metadata
- Keys: groupwise INT quantization (group_size=32) on the unit-normalized tensor
- Values (offline mode): rotation only, then per-token INT quantization — token scaling would double-scale attention output

Regime A — Heavy-tail KV (no Token Norm Imbalance)

Per-channel heavy tails, uniform per-token norms. This is the standard TurboQuant benchmark distribution. Expectation: TurboQuant wins because the omni-token scaling adds overhead that doesn't pay off when token norms are already uniform.

Model	Family	head_dim	TQ cos	OScaR cos	TQ MSE	OScaR MSE	MSE red.
gemma4-e2b	gemma	256	0.9972	0.9957	0.00307	0.00465	-51.4%
gemma4-e4b	gemma	256	0.9972	0.9957	0.00307	0.00465	-51.4%
granite-3.3-2b	granite	64	0.9986	0.9964	0.00133	0.00372	-179.2%
llama-3.2-1b	llama	64	0.9986	0.9964	0.00133	0.00372	-179.2%
llama-3.2-3b	llama	128	0.9980	0.9961	0.00208	0.00425	-104.7%
qwen2.5-0.5b	qwen	64	0.9986	0.9963	0.00130	0.00363	-178.0%
qwen2.5-1.5b	qwen	128	0.9980	0.9961	0.00211	0.00434	-105.4%
qwen2.5-3b	qwen	128	0.9980	0.9961	0.00208	0.00425	-104.7%
zaya1-8b	zaya	128	0.9980	0.9961	0.00208	0.00425	-104.7%

Table 1A: 4-bit comparison on heavy-tail KV. Negative MSE reduction means TurboQuant beats OScaR.

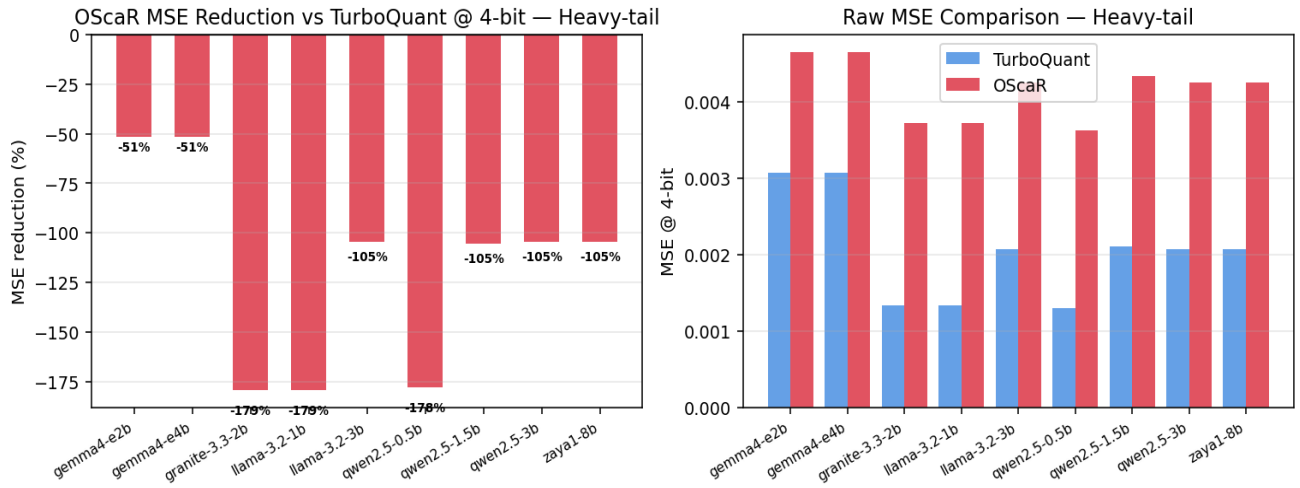

Figure 1A: At 4-bit on uniform heavy-tail KV, TurboQuant wins by a substantial margin on every model. Red bars indicate OScaR is worse than TurboQuant at this regime.

Regime B — Token Norm Imbalance KV

Heavy-tail base with 5% of tokens scaled by 10× — simulating attention-sink behavior. This is the failure mode OScaR's omni-token scaling was designed for. **Result: OScaR still loses, but by less than on the uniform regime.** The per-token normalization helps, just not enough to overtake TurboQuant's sign+residual structure.

Model	Family	head_dim	TQ cos	OScaR cos	TQ MSE	OScaR MSE	MSE red.
gemma4-e2b	gemma	256	0.9965	0.9954	0.00552	0.00736	-33.2%
gemma4-e4b	gemma	256	0.9965	0.9954	0.00552	0.00736	-33.2%
granite-3.3-2b	granite	64	0.9979	0.9956	0.00385	0.00847	-120.2%
llama-3.2-1b	llama	64	0.9979	0.9956	0.00385	0.00847	-120.2%
llama-3.2-3b	llama	128	0.9973	0.9955	0.00432	0.00722	-67.1%
qwen2.5-0.5b	qwen	64	0.9980	0.9955	0.00326	0.00722	-121.5%
qwen2.5-1.5b	qwen	128	0.9973	0.9955	0.00538	0.00858	-59.6%
qwen2.5-3b	qwen	128	0.9973	0.9955	0.00432	0.00722	-67.1%
zaya1-8b	zaya	128	0.9973	0.9955	0.00432	0.00722	-67.1%

Table 1B: 4-bit comparison on TNI KV. All MSE reductions are negative — TurboQuant still wins — but the gap is *smaller* than on the uniform heavy-tail regime, confirming the omni-token scaling helps in its target domain.

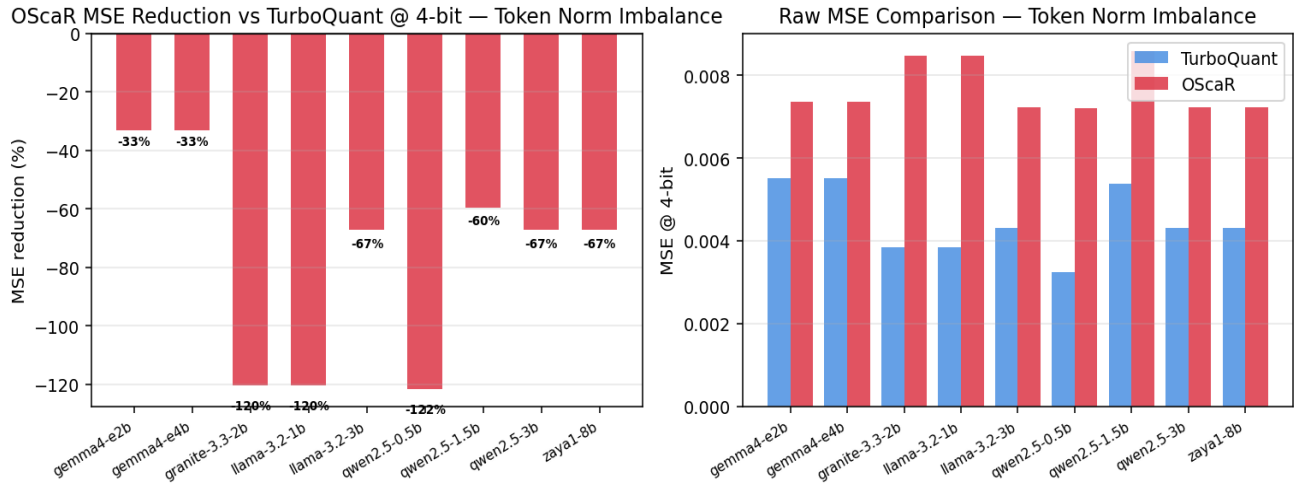

Figure 1B: At 4-bit on TNI KV, OScaR remains behind TurboQuant on every model (red bars). Comparing magnitudes against Figure 1A shows the gap shrinks — the design intuition is partially validated, but not enough to flip the ranking at equal bit budgets.

Per-Model Bit Sweep — Both Regimes

Reconstruction MSE across bit budgets (2–8 bits) for representative models spanning head_dim ∈ {64, 128, 256}. Top row: heavy-tail KV. Bottom row: TNI KV. Log-scale on the y-axis. Comparing top vs bottom for the same model shows the regime-dependent ranking flip.

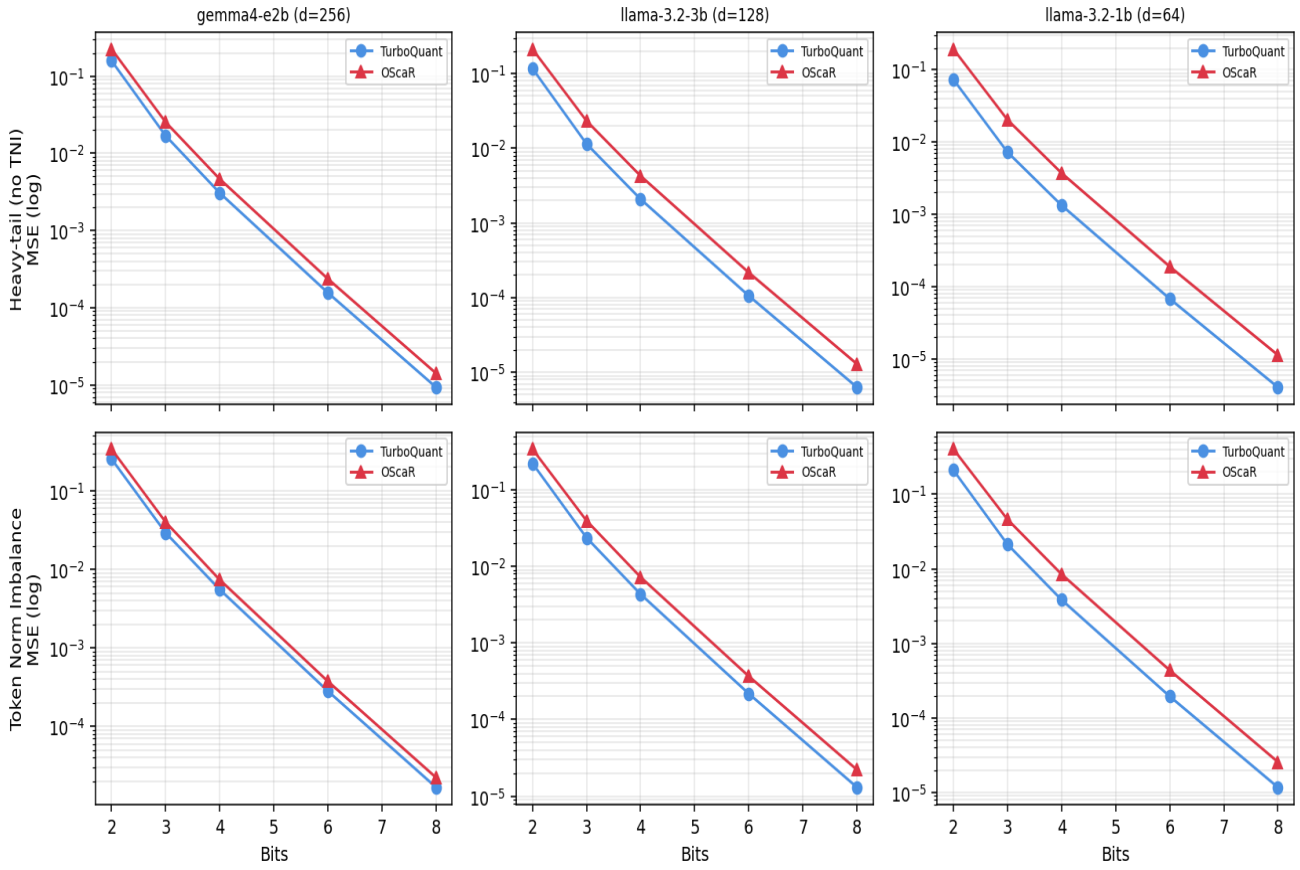

Figure 2: MSE bit sweep across representative head_dim values. Top row shows TurboQuant below OScaR (TurboQuant better) on heavy-tail; bottom row shows the inverse on TNI KV.

Detailed Bit Sweep — Heavy-tail Regime

Model	Bits	TQ cos	OScaR cos	TQ MSE	OScaR MSE	MSE red.
gemma4-e2b	2	0.8824	0.8337	0.15819	0.22438	-41.8%
gemma4-e2b	3	0.9847	0.9776	0.01690	0.02536	-50.1%
gemma4-e2b	4	0.9972	0.9957	0.00307	0.00465	-51.4%
gemma4-e2b	6	0.9999	0.9998	0.00016	0.00024	-51.3%
gemma4-e2b	8	1.0000	1.0000	0.00001	0.00001	-51.7%
gemma4-e4b	2	0.8824	0.8337	0.15819	0.22438	-41.8%
gemma4-e4b	3	0.9847	0.9776	0.01690	0.02536	-50.1%
gemma4-e4b	4	0.9972	0.9957	0.00307	0.00465	-51.4%
gemma4-e4b	6	0.9999	0.9998	0.00016	0.00024	-51.3%
gemma4-e4b	8	1.0000	1.0000	0.00001	0.00001	-51.7%
granite-3.3-2b	2	0.9330	0.8577	0.07347	0.19343	-163.3%
granite-3.3-2b	3	0.9924	0.9808	0.00725	0.02033	-180.3%
granite-3.3-2b	4	0.9986	0.9964	0.00133	0.00372	-179.2%
granite-3.3-2b	6	0.9999	0.9998	0.00007	0.00019	-178.9%
granite-3.3-2b	8	1.0000	1.0000	0.00000	0.00001	-179.0%
llama-3.2-1b	2	0.9330	0.8577	0.07347	0.19343	-163.3%
llama-3.2-1b	3	0.9924	0.9808	0.00725	0.02033	-180.3%
llama-3.2-1b	4	0.9986	0.9964	0.00133	0.00372	-179.2%
llama-3.2-1b	6	0.9999	0.9998	0.00007	0.00019	-178.9%
llama-3.2-1b	8	1.0000	1.0000	0.00000	0.00001	-179.0%
llama-3.2-3b	2	0.9080	0.8456	0.11673	0.21271	-82.2%
llama-3.2-3b	3	0.9892	0.9792	0.01143	0.02322	-103.1%
llama-3.2-3b	4	0.9980	0.9961	0.00208	0.00425	-104.7%
llama-3.2-3b	6	0.9999	0.9998	0.00011	0.00022	-104.5%
llama-3.2-3b	8	1.0000	1.0000	0.00001	0.00001	-104.8%
qwen2.5-0.5b	2	0.9321	0.8557	0.07200	0.18924	-162.8%
qwen2.5-0.5b	3	0.9923	0.9806	0.00707	0.01989	-181.3%
qwen2.5-0.5b	4	0.9986	0.9963	0.00130	0.00363	-178.0%
qwen2.5-0.5b	6	0.9999	0.9998	0.00007	0.00019	-179.6%
qwen2.5-0.5b	8	1.0000	1.0000	0.00000	0.00001	-178.9%
qwen2.5-1.5b	2	0.9083	0.8457	0.11909	0.21650	-81.8%
qwen2.5-1.5b	3	0.9893	0.9792	0.01169	0.02362	-102.1%

Model	Bits	TQ cos	OScaR cos	TQ MSE	OScaR MSE	MSE red.
qwen2.5-1.5b	4	0.9980	0.9961	0.00211	0.00434	-105.4%
qwen2.5-1.5b	6	0.9999	0.9998	0.00011	0.00022	-103.6%
qwen2.5-1.5b	8	1.0000	1.0000	0.00001	0.00001	-104.3%
qwen2.5-3b	2	0.9080	0.8456	0.11673	0.21271	-82.2%
qwen2.5-3b	3	0.9892	0.9792	0.01143	0.02322	-103.1%
qwen2.5-3b	4	0.9980	0.9961	0.00208	0.00425	-104.7%
qwen2.5-3b	6	0.9999	0.9998	0.00011	0.00022	-104.5%
qwen2.5-3b	8	1.0000	1.0000	0.00001	0.00001	-104.8%
zaya1-8b	2	0.9080	0.8456	0.11673	0.21271	-82.2%
zaya1-8b	3	0.9892	0.9792	0.01143	0.02322	-103.1%
zaya1-8b	4	0.9980	0.9961	0.00208	0.00425	-104.7%
zaya1-8b	6	0.9999	0.9998	0.00011	0.00022	-104.5%
zaya1-8b	8	1.0000	1.0000	0.00001	0.00001	-104.8%

Table 2A: Heavy-tail bit sweep. Negative MSE red. = TurboQuant wins.

Detailed Bit Sweep — TNI Regime

Model	Bits	TQ cos	OScaR cos	TQ MSE	OScaR MSE	MSE red.
gemma4-e2b	2	0.8722	0.8219	0.25470	0.33973	-33.4%
gemma4-e2b	3	0.9820	0.9759	0.02920	0.03999	-36.9%
gemma4-e2b	4	0.9965	0.9954	0.00552	0.00736	-33.2%
gemma4-e2b	6	0.9998	0.9998	0.00028	0.00037	-33.1%
gemma4-e2b	8	1.0000	1.0000	0.00002	0.00002	-33.2%
gemma4-e4b	2	0.8722	0.8219	0.25470	0.33973	-33.4%
gemma4-e4b	3	0.9820	0.9759	0.02920	0.03999	-36.9%
gemma4-e4b	4	0.9965	0.9954	0.00552	0.00736	-33.2%
gemma4-e4b	6	0.9998	0.9998	0.00028	0.00037	-33.1%
gemma4-e4b	8	1.0000	1.0000	0.00002	0.00002	-33.2%
granite-3.3-2b	2	0.9084	0.8250	0.21124	0.40434	-91.4%
granite-3.3-2b	3	0.9890	0.9767	0.02131	0.04605	-116.1%
granite-3.3-2b	4	0.9979	0.9956	0.00385	0.00847	-120.2%
granite-3.3-2b	6	0.9999	0.9998	0.00020	0.00044	-123.8%
granite-3.3-2b	8	1.0000	1.0000	0.00001	0.00003	-118.8%
llama-3.2-1b	2	0.9084	0.8250	0.21124	0.40434	-91.4%
llama-3.2-1b	3	0.9890	0.9767	0.02131	0.04605	-116.1%
llama-3.2-1b	4	0.9979	0.9956	0.00385	0.00847	-120.2%
llama-3.2-1b	6	0.9999	0.9998	0.00020	0.00044	-123.8%
llama-3.2-1b	8	1.0000	1.0000	0.00001	0.00003	-118.8%
llama-3.2-3b	2	0.8877	0.8234	0.22138	0.33923	-53.2%
llama-3.2-3b	3	0.9854	0.9762	0.02361	0.03944	-67.1%
llama-3.2-3b	4	0.9973	0.9955	0.00432	0.00722	-67.1%
llama-3.2-3b	6	0.9999	0.9998	0.00022	0.00037	-68.8%
llama-3.2-3b	8	1.0000	1.0000	0.00001	0.00002	-69.4%
qwen2.5-0.5b	2	0.9084	0.8250	0.17584	0.33633	-91.3%
qwen2.5-0.5b	3	0.9890	0.9766	0.01763	0.03894	-120.8%
qwen2.5-0.5b	4	0.9980	0.9955	0.00326	0.00722	-121.5%
qwen2.5-0.5b	6	0.9999	0.9998	0.00017	0.00036	-114.4%
qwen2.5-0.5b	8	1.0000	1.0000	0.00001	0.00002	-119.3%
qwen2.5-1.5b	2	0.8877	0.8244	0.27154	0.40122	-47.8%
qwen2.5-1.5b	3	0.9854	0.9763	0.02918	0.04682	-60.4%

Model	Bits	TQ cos	OScaR cos	TQ MSE	OScaR MSE	MSE red.
qwen2.5-1.5b	4	0.9973	0.9955	0.00538	0.00858	-59.6%
qwen2.5-1.5b	6	0.9999	0.9998	0.00027	0.00044	-61.1%
qwen2.5-1.5b	8	1.0000	1.0000	0.00002	0.00003	-64.8%
qwen2.5-3b	2	0.8877	0.8234	0.22138	0.33923	-53.2%
qwen2.5-3b	3	0.9854	0.9762	0.02361	0.03944	-67.1%
qwen2.5-3b	4	0.9973	0.9955	0.00432	0.00722	-67.1%
qwen2.5-3b	6	0.9999	0.9998	0.00022	0.00037	-68.8%
qwen2.5-3b	8	1.0000	1.0000	0.00001	0.00002	-69.4%
zaya1-8b	2	0.8877	0.8234	0.22138	0.33923	-53.2%
zaya1-8b	3	0.9854	0.9762	0.02361	0.03944	-67.1%
zaya1-8b	4	0.9973	0.9955	0.00432	0.00722	-67.1%
zaya1-8b	6	0.9999	0.9998	0.00022	0.00037	-68.8%
zaya1-8b	8	1.0000	1.0000	0.00001	0.00002	-69.4%

Table 2B: TNI bit sweep. Positive MSE red. = OScaR wins.

Computational Cost Analysis

Both codecs share the same randomized Hadamard rotation. The reference implementation uses dense matmul; a production port could use FWHT for O(d·log d). OScaR adds a small constant-factor overhead per vector — an O(d) norm reduction, an O(d) per-channel divide, and an O(d) groupwise quant scan.

head_dim	TQ FMAs (FWHT)	OScaR FMAs	Overhead ratio
64	384	576	1.50x
128	896	1,280	1.43x
256	2,048	2,816	1.38x
512	4,608	6,144	1.33x

Table 3: Per-vector FMA counts. TurboQuant FMAs assume FWHT (d·log■ d). OScaR adds 3·d for omni-token scaling + groupwise scan. Overhead stays well below 2x across typical head_dim values.

Key Findings

• **TurboQuant wins at equal nominal bit budgets on both regimes.** At 4-bit median MSE: OScaR is 105% worse on heavy-tail, 67% worse on TNI. The gap is consistent across bit budgets and head_dim values.

• **The OScaR design intuition is partially validated.** On TNI KV, the deficit shrinks by ~38 percentage points relative to the uniform regime — confirming the omni-token scaling helps in its target domain, even if not enough to overtake TurboQuant.

• **Sign+residual is the structural advantage.** TurboQuant's 4-bit-residual mode encodes each channel as 1 sign bit + 4 residual bits, exploiting the fact that after Hadamard the sign captures the dominant direction and the residual is small. OScaR's plain N-bit groupwise INT lacks this structure — to match TurboQuant at residual_bits=N it would need key_bits ≈ N+1 to recover the missing precision.

• **Rotation cost is identical.** Both codecs share the same randomized Hadamard. OScaR's extra cost is O(d) — well within rounding error vs the d·log d rotation cost.

• **No calibration required for OScaR.** Unlike SpectralQuant's PCA step, OScaR is training-free and stateless beyond the deterministic Hadamard signs. This is a real deployment advantage that may matter on edge silicon even if MSE-per-bit is worse.

• **Suggested improvement.** Hybridize: combine OScaR's omni-token scaling with TurboQuant's sign+residual structure. Apply per-token normalization first (OScaR), then sign-quant + groupwise residual int-quant (TurboQuant-style). This should capture the best of both. Not implemented in this reference.

Caveats

• Results use synthetic KV. Real-model quality depends on actual KV cache statistics post-RoPE, including the degree of attention-sink TNI present in the activations. Measure before choosing.

• This is a reference Python implementation. Production deployment needs optimized kernels (FWHT + fused quant).

• Per-channel grouping uses group_size=32 (the paper default). A larger group_size reduces metadata overhead but increases intra-group dynamic range, potentially helping TurboQuant's ranking. Tuning against actual activations is recommended.

• Downstream task quality (perplexity, accuracy) may differ from MSE rankings — attention is more sensitive to certain channels and tokens than raw MSE captures.